

Language Technology

Chapter 16: Transformers: Pretraining an Encoder

https://link.springer.com/chapter/10.1007/978-3-031-57549-5_16

Pierre Nugues

Pierre.Nugues@cs.lth.se

October 8 and 15, 2026



Training Encoders

Encoders, such as BERT, are often trained on masked language models with two tasks:

- 1 For a sentence, predict masked words: We replace 15% of the tokens with a specific mask token and we train the model to predict them. This is just a cloze test;
- 2 For a pair of sentences, predict if the second one is the successor of the first one;

Taking the two first sentences from the *Odyssey*:

*Tell me, O Muse, of that ingenious hero who travelled far and wide after he had sacked the famous town of Troy.
Many cities did he visit, and many were the nations with whose manners and customs he was acquainted;*



Masked language models

We add two special tokens: [CLS] at the start of the first sentence and [SEP] at the end of both sentences, and the token [MASK] to denote the words to predict.

We would have for the first task:

[CLS] Tell me, O Muse, of that [MASK] hero who travelled far and wide [MASK] he had sacked the [MASK] town of Troy. [SEP]

For the second task, we would have as input:

[CLS] Tell me, O Muse, of that [MASK] hero who travelled far and wide [MASK] he had sacked the [MASK] town of Troy. [SEP] Many cities did he [MASK visit], and many were the [MASK nations] with whose manners [MASK and] customs he was acquainted; [SEP]

where the prediction would return that the second sentence is the next one (as opposed to random sequences)



Positional Embeddings

BERT (the first transformer, Devlin et al. (2019)) maps each token to three embedding vectors:

- the token embedding,
- the position of the token in the sentence (positional embeddings), and
- the segment embeddings (we will skip this part).

The three kinds of embeddings are learnable vectors.

In the BERT base version, each embedding vector has 768 dimensions.

Let us consider two sentences simplified from the *Odyssey*:

Tell me of that hero. Many cities did he visit.

Input:	[CLS]	Tell	me	of	that	hero	[SEP]	Many	cities	did	he	visit	[SEP]
Token	$E_{[CLS]}$	E_{tell}	E_{me}	E_{of}	E_{that}	E_{hero}	$E_{[SEP]}$	E_{many}	E_{cities}	E_{did}	E_{he}	E_{visit}	$E_{[SEP]}$
Segment	E_A	E_A	E_A	E_A	E_A	E_A	E_A	E_B	E_B	E_B	E_B	E_B	E_B
Position	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}	E_{11}	E_{12}



Code Example

Jupyter Notebook:

<https://github.com/pnugues/pnlp/tree/main/notebooks>



Model size

Transformers are trained on large corpora like the colossal clean crawled corpus (<https://arxiv.org/abs/2104.08758>) and encapsulate semantics found in text in the form of numerical matrices.

This results in large models (Devlin et al., 2019):

In this work, we denote the number of layers (i.e., Transformer blocks) as L , the hidden size as H , and the number of self-attention heads as A . We primarily report results on two model sizes: $BERT_{BASE}$ ($L=12$, $H=768$, $A=12$, Total Parameters=110M) and $BERT_{LARGE}$ ($L=24$, $H=1024$, $A=16$, Total Parameters=340M).

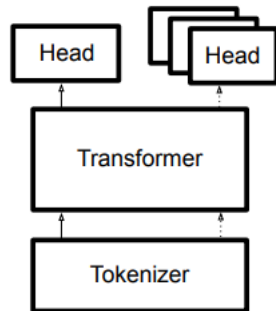
Transformers can then act as pre-trained models for a variety of tasks. See the list from Hugging face

Finally, an interesting reading: <https://sayakpaul.medium.com/an-interview-with-colin-raffel-research-scientist-at-google-5>



Applying Transformers

- Matrices in the transformer architecture encapsulate massive knowledge from text.
- We can apply them to tasks beyond what they have been trained for (masked language model)
- We use them as pretrained models and fine-tune a so-called dedicated *head*.
- The simplest head is a logistic regression

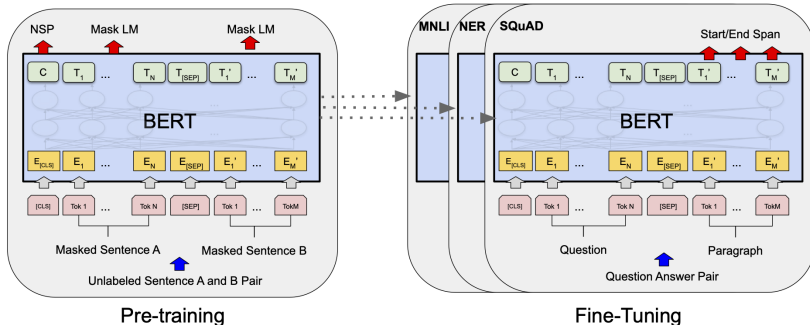


Picture from Wolf et al., Transformers: State-of-the-art Natural Language Processing, EMNLP Demos 2020.



Transfer Learning

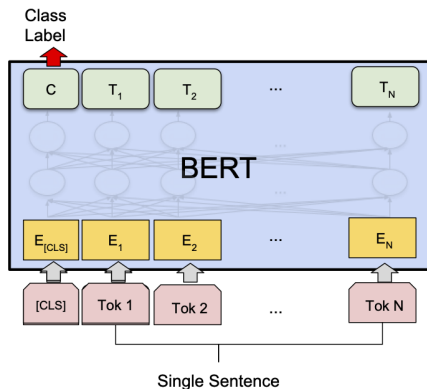
Transfer learning consists of a costly **pretraining** step and an adaptation to applications called **fine-tuning**.



from Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, 2019. Note the picture comes from the second version of the paper from 2019



Application: Sentence Classification



(b) Single Sentence Classification Tasks:
SST-2, CoLA

from Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, 2019



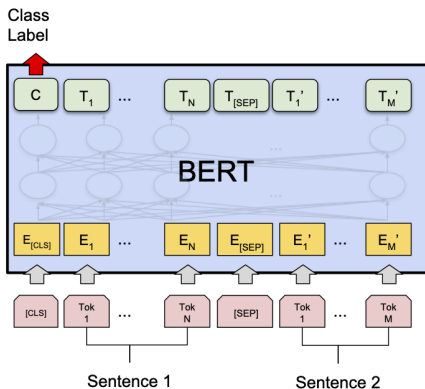
Code Example

Jupyter Notebook:

<https://github.com/pnugues/pnlp/tree/main/notebooks>



Application: Sentence Pair Classification

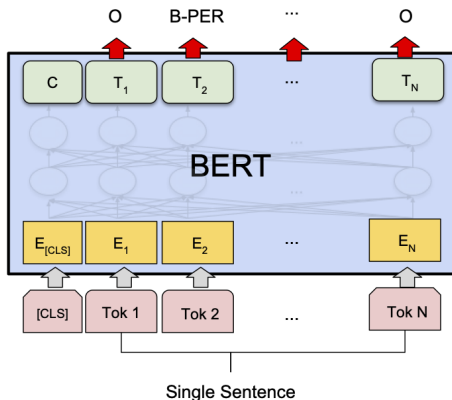


(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG

from Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, 2019



Application: Sequence Tagging



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

from Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, 2019.



Stanford Question Answering Dataset (SQuAD)

- Consists of 100,000 questions and paragraphs from wikipedia containing the answers
- The answer is a segment in the text (factoid QA)
- Complemented by SQuAD 2.0 with unanswerable questions
- SQuAD started an intense competition. See the impressive leaderboard

<https://rajpurkar.github.io/SQuAD-explorer/>

Form Rajpurkar et al., *SQuAD: 100,000+ Questions for Machine Comprehension of Text*, 2016

In meteorology, precipitation is any product of the condensation of atmospheric water vapor that falls under **gravity**. The main forms of precipitation include drizzle, rain, sleet, snow, **graupel** and hail... Precipitation forms as smaller droplets coalesce via collision with other rain drops or ice crystals **within a cloud**. Short, intense periods of rain in scattered locations are called "showers".

What causes precipitation to fall?

gravity

What is another main form of precipitation besides drizzle, rain, snow, sleet and hail?

graupel

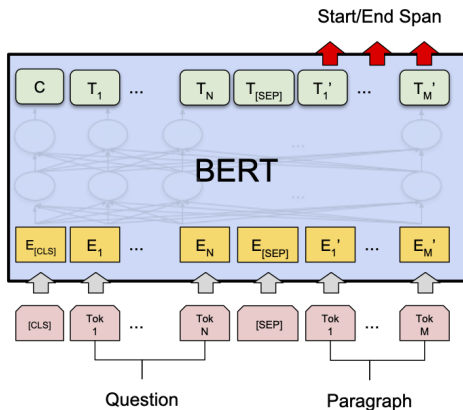
Where do water droplets collide with ice crystals to form precipitation?

within a cloud

Figure 1: Question-answer pairs for a sample passage in the SQuAD dataset. Each of the answers is a segment of text from the passage.



Application: Question Answering



(c) Question Answering Tasks:
SQuAD v1.1

from Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, 2019.



Modern Alternatives

- BERT has proven very effective in classification and sequence annotation:
 - GLUE
 - SuperGlue (<https://gluebenchmark.com/>)
- BERT is an autoencoder somewhat overshadowed by autoregressive models (decoders such as GPT-2 and its successors)
- Newer versions integrate LLM progress. See ModernBERT (<https://huggingface.co/blog/modernbert>)
- New encoder models, building on BERT, evolved to work in an unsupervised manner: Laya and the likes (<https://github.com/NandhaKishorM/laya>)

